

Revenue & ROAS Forecasting

A Plain-Language Guide to Our Project

Algnition 3.0 | NetElixir Hackathon

TOP 10 FINALIST

What we built, how it works, and exactly which file does what.

Team	Sigmoid
Member	Rehaan Ahmad Khan
College	JSS University, Noida
Repository	github.com/RAK2315/netElixir

~8%

typical error on total sales

80% band

low-high safety range

30/60/90 days

how far ahead

3 ad channels

Google, Bing, Meta

This document is written in simple language so anyone on the team can understand the whole project and explain it confidently in the finalist presentation.

1. What This Project Is (in plain words)

Online stores spend money on ads across Google, Microsoft/Bing and Meta (Facebook/Instagram). Before they spend, they want to know: how much money will these ads bring back over the next 1 to 3 months, and will it be worth it?

Our project answers that. You give it past ad data; it predicts future SALES (revenue) and how efficient the spending is (ROAS). Instead of one risky guess, it gives a low, middle and high estimate (like a weather forecast). It also lets you play with budgets ('what if I spend more on Meta?') and uses an AI to explain the forecast in normal English.

The one-line version

Feed in past ad data, get back a realistic low-middle-high forecast of future sales and ad efficiency for each campaign, plus an AI-written explanation of why.

2. The Problem We Were Asked to Solve

Marketing agencies must promise results before spending. That is hard because sales change with seasons, customer behaviour, and messy data spread across different ad platforms. The hackathon asked us to build a practical tool that:

- predicts future revenue and ROAS as a low-middle-high range, not a single number;
- breaks the forecast down by channel, campaign type, and individual campaign;
- works for 30, 60 and 90-day plans and can test different budgets;
- uses the ad platforms' own numbers as-is (we don't rebuild their tracking); and
- uses an AI to explain the forecast and flag risks.

3. The Data We Were Given

Three files of daily ad data, one per channel. They look different and one has a trap:

- Google - biggest file (about 19,000 rows, 92 campaigns). Its cost is stored in 'micros' (millionths), so we divide by a million to get real money.
- Bing - smaller (about 2,900 rows). Clean revenue and spend.
- Meta - about 3,400 rows, BUT it has NO revenue column. It only has a 'conversion' count that adds up to ~1.66 million, which is clearly not real purchases.

The Meta trap (and our smart fix)

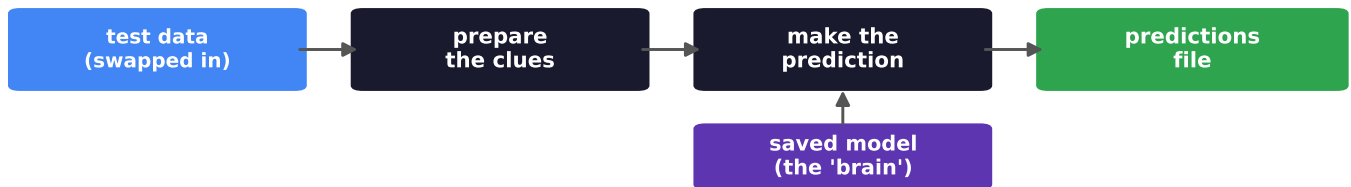
If we naively multiplied Meta's 1.66 million 'conversions' by an average order value, we'd invent about \$146 million of fake sales. Instead we estimate Meta revenue as its spend times the real return rate seen on Google and Bing (about 4.75x). This keeps Meta sensible and honest, and we clearly label it as an estimate. Many teams will miss this.

4. The Big Idea: Two Separate Parts

The judges test the project in two ways, so we built it in two clean parts:

- Part A - The 'scoring engine'. A robot pipeline downloads our code, swaps in its own secret test data, runs ONE command, and checks our predictions. It runs with NO internet. If anything breaks, we score zero. This part is simple, fast and reliable.
- Part B - The 'showcase'. An interactive dashboard and the AI explanations. This is for humans to explore. It can use the internet, but it NEVER runs inside the scored command, so it can never cause the scoring to fail.

PART A - Scoring engine (offline, always works)



PART B - Showcase (for humans, not scored)



5. How the Prediction Works, Step by Step

In plain terms, the tool does four things in order:

- STEP 1 - Tidy up. Read the three messy files and merge them into one clean table. Fix units, estimate Meta's revenue, and handle bad values.
- STEP 2 - Make clues. For each campaign, summarise its recent behaviour (how much it spent and earned lately, its trend, the season) into numbers the model can learn from. These are 'features'.
- STEP 3 - Predict a range. A trained model looks at the clues and outputs a low, middle and high revenue figure for the next 30, 60 or 90 days. ROAS is just revenue divided by spend.
- STEP 4 - Explain. The AI reads the numbers and writes a short, plain-English briefing: what to expect, why, the risks, and what to do.

6. Key Words, Explained Simply

ROAS

'Return On Ad Spend'. For every \$1 spent on ads, how many dollars of sales come back. ROAS of 4 means *4backforevery1* spent. Higher is better.

Revenue

The total sales money the ads bring in. Our main thing to predict.

Probabilistic range (low / middle / high)

Instead of one exact guess, we give three numbers. The middle is our best guess; the low and high show how uncertain we are. Like saying 'tomorrow around 20 degrees, likely between 15 and 25'.

P10 / P50 / P90

Fancy names for those three. P50 = middle. P10 = cautious low. P90 = optimistic high. About 80% of the time the real answer lands between P10 and P90.

Forecast window (30/60/90 days)

How far into the future we predict - one, two, or three months.

Run-rate

'Keep going at your current pace.' Unless you change the budget, we assume spending continues at the recent average.

Quantile model (LightGBM)

A fast, proven prediction method. We train three of them - one for the low line, one for the middle, one for the high line.

Conformal calibration

An honesty check. We test our low-to-high range on past data; if it was too narrow (missed too often), we widen it so it's trustworthy.

WAPE (our accuracy score)

How far off our prediction was, as a percentage. Our ~8% means our total-sales forecast was off by only about 8% on average - quite accurate.

7. What Each File Does (the important part)

If you open the project on GitHub, here is what every main file is for, in plain words.

The scoring engine (Part A - what the judges run)

`run.sh`

The single 'start button'. The judges run just this one command. It runs the two steps below in order and writes the predictions file.

`src/common/schema.py`

The clean-up crew. Reads the three different ad files, merges them into one tidy table, fixes units, and does the smart Meta revenue estimate.

`src/forecasting/features.py`

The clue-maker. Turns each campaign's history into the summary numbers ('features') the model learns from.

`src/forecasting/model.py`

The brain. Takes the clues and produces the low/middle/high revenue and ROAS forecasts, and adds up campaigns into channel and total forecasts.

`src/forecasting/conformal.py`

The honesty check that keeps the low-high range reliable.

`src/generate_features.py`

Step 1 of the run: prepares the clues from whatever data is given.

`src/predict.py`

Step 2 of the run: uses the brain to write the final predictions.csv file.

`src/train.py`

How we taught the brain, using the sample data. We run this once ourselves; the judges do NOT - they use the already-trained brain.

`pickle/model.pkl`

The saved, trained brain - a single file the judges load and use.

The showcase (Part B - for humans)

`app/streamlit_app.py`

The interactive dashboard: charts, budget sliders, and the AI panel.

`src/insights.py`

Asks an AI (Groq) to explain the forecast in plain English. If there's no internet or key, it writes a sensible explanation itself, so it always works.

Supporting files

`requirements.txt`

The exact list of tools (and versions) needed to run the scoring engine.

`data/`

Sample ad data so the project runs out of the box. The judges swap in their own.

`tests/test_pipeline.py`

Automatic checks that confirm everything still works correctly.

`docs/`

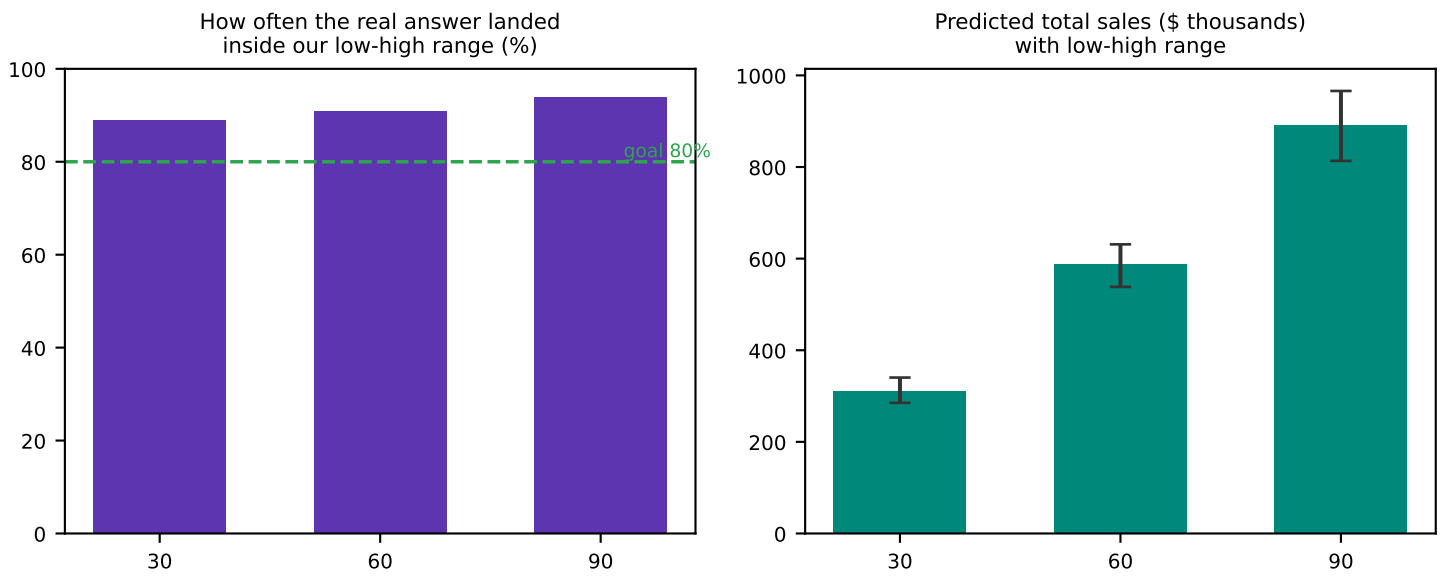
The written explanations, including this report and the methodology/architecture notes.

`README.md`

The front-page summary and quick instructions.

8. How Accurate Are We?

We tested honestly: we trained on older data and checked predictions on newer data the model had never seen (a fair 'exam').



- On TOTAL sales, our middle guess was off by only about 8% on average - the number a business actually cares about.
- Our low-high range caught the real answer about 89-94% of the time (goal was 80%), so the range is honest and a little on the safe side.

9. The File We Hand In

Following the organizers' instruction ('same columns as the input data plus the forecast'), the predictions file has one row per campaign per time window, with these columns:

```
channel, campaign_id, campaign_name, campaign_type, horizon_days,
Revenue, ROAS, Revenue_p10, Revenue_p90, ROAS_p10, ROAS_p90
```

Revenue and ROAS are the middle (best) guess; the _p10 and _p90 columns are the low and high ends of the range.

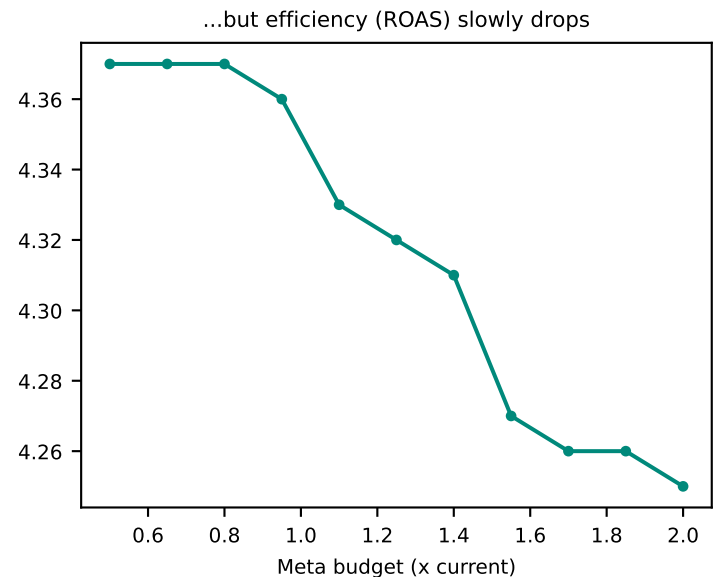
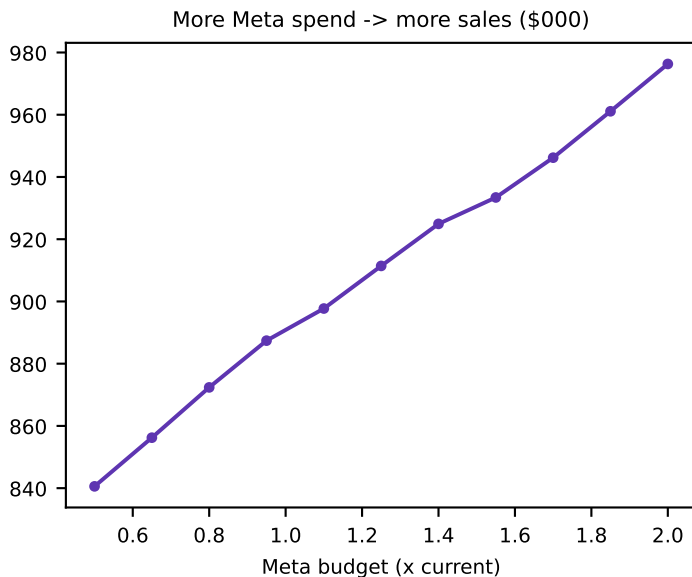
10. The AI Explanation Part

After the numbers are ready, an AI writes a short briefing in four parts: what to expect, why (the causes), the risks, and what to do next. Two things make it trustworthy:

- It only explains OUR numbers - it is never allowed to invent figures.
- If there's no internet or AI key, the tool writes a sensible explanation itself, so a demo never fails. The scoring engine never uses AI at all.

11. The Dashboard (the 'wow' for judges)

A simple web page where you can slide a channel's budget up or down and instantly see the sales and ROAS forecast change. Below: as we spend more on Meta, predicted sales rise but efficiency (ROAS) slowly drops - the classic 'more money brings less extra return' story.



12. Making Sure It Never Breaks

Because the judges feed in their own data, we tried to break our own tool with 30 crazy situations - empty files, weird symbols where numbers should be, strange dates, a missing channel, emojis in names, huge and tiny numbers, and more. All 30 were handled without crashing. A crash would score zero, so this reliability matters a lot.

13. How to Run It Yourself

The scoring engine (what the judges run)

```
pip install -r requirements.txt
./run.sh ./data ./pickle/model.pkl ./output/predictions.csv
```

The dashboard

```
pip install -r requirements.txt -r requirements-app.txt
streamlit run app/streamlit_app.py
```

14. Simple Points to Make in the Presentation

- We predict a realistic RANGE, not one risky number - honest about uncertainty.
- We caught the Meta 'no revenue' trap and fixed it sensibly - shows real understanding.
- Our forecast of total sales is accurate to about 8%.
- It reliably passes the strict offline test - many projects fail there.
- You can play with budgets live and get an AI explanation in plain English.

In short: a genuinely useful, honest, and reliable forecasting tool - not just a demo.

Part 2 - Exactly How It Was Built

Part 1 explained the ideas in plain words. Part 2 documents precisely what each piece does and how, file by file, including the actual settings we used. This is the technical record of the work.

A. Data preprocessing (src/common/schema.py)

Every raw file passes through one module that outputs a single unified table.

- Channel auto-detection: each file is identified by its set of column names (a 'signature'), NOT its filename - so the judges' renamed files still load.
- Unit fixes: Google cost is in micros, so $\text{spend} = \text{metrics_cost_micros} / 1,000,000$. Google revenue = $\text{metrics_conversions_value}$; Bing uses its native Revenue/Spend.
- Campaign-type normalisation: 'PerformanceMax', 'PERFORMANCE_MAX', 'Pmax' all map to one label, so channel-agnostic breakdowns are consistent.
- Robust parsing: numbers with commas or junk ('1,234.5', '-', 'N/A') are coerced safely; bad or mixed-format dates become empty and are dropped; negative metrics are clipped to 0; duplicate (channel, campaign, date) rows are de-duplicated (keep last).

The Meta revenue estimate (the key preprocessing decision)

```
blended_ROAS = sum(revenue) / sum(spend)      # over Google + Bing only (~4.75)
meta_revenue = meta_spend * blended_ROAS      # Meta has no revenue column
```

Meta's own 'conversion' field is deliberately NOT used for revenue (it is a broad metric that sums to ~1.66M). Every Meta row is flagged `revenue_is_imputed = True`.

B. Turning history into 'clues' (src/forecasting/features.py)

We frame forecasting as ordinary table-based machine learning using a 'direct multi-horizon' design. For each campaign and each cut-off date, we build one training row that asks: given what we know up to date t , how much revenue in the next H days?

- Cut-off dates are swept every 14 days across history; a campaign needs 90 days of history behind the cut-off and the full future window ahead to form a valid row.
- Speed trick: each campaign's daily series is laid on a full calendar once and cumulative sums are pre-computed, so any trailing or future window sum is instant.
- No leakage: every input uses data on/before t ; the answer (label) uses only the future window ($t, t+H$].

The exact features the model sees

```
trailing spend / revenue / conversions over 7, 14, 30, 60, 90 days
trailing ROAS over 30 and 90 days
active_days_30, active_days_90, days_since_start, days_since_last_active
spend_trend_30_60, revenue_trend_30_60, revenue_vol_90, runrate_daily_spend_30
planned_spend, planned_vs_runrate, horizon (30/60/90)
fc_month_sin, fc_month_cos, fc_quarter    (season of the forecast window)
channel, campaign_type    (categorical)
```

At prediction time 'planned_spend' defaults to the recent run-rate x horizon; the budget simulator simply scales it. Only currently-active campaigns are forecast, with a graceful fall-back (30 -> 90 days -> all-time) so a quiet period never returns nothing.

C. The model & training (model.py, conformal.py, train.py)

Three gradient-boosted models predict the low, middle and high of window revenue.

```
LightGBM (native Booster - NO scikit-learn dependency)
objective = 'quantile', alpha = 0.1 / 0.5 / 0.9    (three models)
num_boost_round=500, learning_rate=0.03, num_leaves=31,
min_data_in_leaf=30, bagging_fraction=0.8, feature_fraction=0.8,
lambda_l2=1.0, seed=42    (categorical: channel, campaign_type)
```

Using the native Booster (not the scikit-learn wrapper) means the saved model needs zero scikit-learn to load - avoiding the single most common cause of submission failure.

Honest ranges: Conformalized Quantile Regression

```
split by time: 65% train | 15% calibration | 20% test (most recent)
on calibration set: E = max(p10 - y, y - p90)        per row
widen each side by the (1 - alpha) empirical quantile of E, per horizon
then enforce p10 <= p50 <= p90
```

This calibration is why our measured coverage (89-94%) lands at/above the 80% goal. After testing, the three models are re-fit on ALL data for the shipped model.pkl, keeping the horizon-level widening.

Adding campaigns up to totals (coherent ranges)

Channel / type / blended ranges are NOT a naive sum of quantiles. We draw samples for each campaign consistent with its (p10, p50, p90), sum them, and read off the combined percentiles (Monte-Carlo, fixed seed). Point totals are exact sums of the P50s.

D. The scored run & output (run.sh, generate_features.py)

- run.sh sets PYTHONPATH, accepts DATA_DIR / MODEL_PATH / OUTPUT_PATH (with defaults), and runs two Python steps end-to-end under 'set -e || pipefail'.
- generate_features.py reads whatever CSVs are in data/, unifies them, builds one feature row per active campaign x horizon, and writes features.parquet.

- predict.py loads model.pkl, predicts the three quantiles, computes ROAS = revenue / planned spend, and writes the output in the organizer's format:

```
channel, campaign_id, campaign_name, campaign_type, horizon_days,  
Revenue, ROAS, Revenue_p10, Revenue_p90, ROAS_p10, ROAS_p90
```

E. Reliability & reproducibility (how we made it safe)

- Fuzz-tested 30 extreme inputs through the full path; empty/no-spend data yields a valid empty file instead of crashing.
- Seeds fixed everywhere; no absolute paths; no network at run time; versions pinned so the model always unpickles.
- Verified by an actual 'git clone' into a fresh virtual environment containing only the four core packages (no scikit-learn) - it produced a valid predictions file.

That is the complete build: messy data in, one clean offline command, an honest probabilistic forecast out, and a human-friendly showcase on top.